МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:
А.С. Кот
Студент группы ПО-13
Проверил:
А.Д. Кулик
преп.-стажёр кафедры ИИТ

Брест 2026

Цель: приобрести практические навыки разработки API и баз данных.
Задание:
1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;
Вариант: 10) База данных Учет успеваемости

database.py

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql://postgres:@localhost/academic_performance"

engine = create_engine(DATABASE_URL, echo=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

models.py

```
from sqlalchemy import Column, Integer, String, Date, ForeignKey
from sqlalchemy.orm import relationship
from database import Base

class Student(Base):
    __tablename__ = 'students'
    student_id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    enrollment_date = Column(Date, nullable=False)
    group_name = Column(String(20), nullable=False)
    grades = relationship("Grade", back_populates="student")
    attendances = relationship("Attendance", back_populates="student")

class Teacher(Base):
    __tablename__ = 'teachers'
    teacher_id = Column(Integer, primary_key=True, index=True)
    first_name = Column(String(50), nullable=False)
    last_name = Column(String(50), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
    hire_date = Column(Date, nullable=False)
    department = Column(String(100), nullable=False)
    subjects = relationship("Subject", back_populates="teacher")

class Subject(Base):
    __tablename__ = 'subjects'
    subject_id = Column(Integer, primary_key=True, index=True)
    subject_name = Column(String(100), unique=True, nullable=False)
    credits = Column(Integer, nullable=False)
    teacher_id = Column(Integer, ForeignKey('teachers.teacher_id'))
    teacher = relationship("Teacher", back_populates="subjects")
```

```
    grades = relationship("Grade", back_populates="subject")
    attendances = relationship("Attendance", back_populates="subject")

class Grade(Base):
    __tablename__ = 'grades'
    grade_id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey('students.student_id'))
    subject_id = Column(Integer, ForeignKey('subjects.subject_id'))
    grade = Column(Integer, nullable=False)
    grade_date = Column(Date, nullable=False)
    semester = Column(Integer, nullable=False)
    student = relationship("Student", back_populates="grades")
    subject = relationship("Subject", back_populates="grades")

class Attendance(Base):
    __tablename__ = 'attendance'
    attendance_id = Column(Integer, primary_key=True, index=True)
    student_id = Column(Integer, ForeignKey('students.student_id'))
    subject_id = Column(Integer, ForeignKey('subjects.subject_id'))
    attendance_date = Column(Date, nullable=False)
    status = Column(String(20), nullable=False)
    student = relationship("Student", back_populates="attendances")
    subject = relationship("Subject", back_populates="attendances")
```

schemas.py

```
from pydantic import BaseModel, EmailStr
from datetime import date
from typing import Optional

class StudentBase(BaseModel):
    first_name: str
    last_name: str
    email: EmailStr
    enrollment_date: date
    group_name: str

class StudentCreate(StudentBase):
    pass

class Student(StudentBase):
    student_id: int
    class Config:
        from_attributes = True

class TeacherBase(BaseModel):
    first_name: str
    last_name: str
    email: EmailStr
    hire_date: date
    department: str

class TeacherCreate(TeacherBase):
    pass

class Teacher(TeacherBase):
    teacher_id: int
    class Config:
        from_attributes = True

class SubjectBase(BaseModel):
    subject_name: str
    credits: int
    teacher_id: Optional[int] = None

class SubjectCreate(SubjectBase):
```

```
    pass

class Subject(SubjectBase):
    subject_id: int
    class Config:
        from_attributes = True

class GradeBase(BaseModel):
    student_id: int
    subject_id: int
    grade: int
    grade_date: date
    semester: int

class GradeCreate(GradeBase):
    pass

class Grade(GradeBase):
    grade_id: int
    class Config:
        from_attributes = True

class AttendanceBase(BaseModel):
    student_id: int
    subject_id: int
    attendance_date: date
    status: str

class AttendanceCreate(AttendanceBase):
    pass

class Attendance(AttendanceBase):
    attendance_id: int
    class Config:
        from_attributes = True
```

crud.py

```
from sqlalchemy.orm import Session
import models
import schemas

# ========== Students ==========
def get_students(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Student).offset(skip).limit(limit).all()

def get_student(db: Session, student_id: int):
    return db.query(models.Student).filter(models.Student.student_id ==
student_id).first()

def create_student(db: Session, student: schemas.StudentCreate):
    db_student = models.Student(**student.model_dump())
    db.add(db_student)
    db.commit()
    db.refresh(db_student)
    return db_student

def update_student(db: Session, student_id: int, student: schemas.StudentCreate):
    db_student = get_student(db, student_id)
    if db_student:
        for key, value in student.model_dump().items():
            setattr(db_student, key, value)
        db.commit()
        db.refresh(db_student)
    return db_student
```

```
def delete_student(db: Session, student_id: int):
    db_student = get_student(db, student_id)
    if db_student:
        db.delete(db_student)
        db.commit()
        return True
    return False

# ========== Teachers ==========
def get_teachers(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Teacher).offset(skip).limit(limit).all()

def get_teacher(db: Session, teacher_id: int):
    return db.query(models.Teacher).filter(models.Teacher.teacher_id ==
teacher_id).first()

def create_teacher(db: Session, teacher: schemas.TeacherCreate):
    db_teacher = models.Teacher(**teacher.model_dump())
    db.add(db_teacher)
    db.commit()
    db.refresh(db_teacher)
    return db_teacher

def update_teacher(db: Session, teacher_id: int, teacher: schemas.TeacherCreate):
    db_teacher = get_teacher(db, teacher_id)
    if db_teacher:
        for key, value in teacher.model_dump().items():
            setattr(db_teacher, key, value)
        db.commit()
        db.refresh(db_teacher)
    return db_teacher

def delete_teacher(db: Session, teacher_id: int):
    db_teacher = get_teacher(db, teacher_id)
    if db_teacher:
        db.delete(db_teacher)
        db.commit()
        return True
    return False

# ========== Subjects ==========
def get_subjects(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Subject).offset(skip).limit(limit).all()

def get_subject(db: Session, subject_id: int):
    return db.query(models.Subject).filter(models.Subject.subject_id ==
subject_id).first()

def create_subject(db: Session, subject: schemas.SubjectCreate):
    db_subject = models.Subject(**subject.model_dump())
    db.add(db_subject)
    db.commit()
    db.refresh(db_subject)
    return db_subject

def update_subject(db: Session, subject_id: int, subject: schemas.SubjectCreate):
    db_subject = get_subject(db, subject_id)
    if db_subject:
        for key, value in subject.model_dump().items():
            setattr(db_subject, key, value)
        db.commit()
        db.refresh(db_subject)
    return db_subject

def delete_subject(db: Session, subject_id: int):
```

```
    db_subject = get_subject(db, subject_id)
    if db_subject:
        db.delete(db_subject)
        db.commit()
        return True
    return False

# ========== Grades ==========
def get_grades(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Grade).offset(skip).limit(limit).all()

def get_grade(db: Session, grade_id: int):
    return db.query(models.Grade).filter(models.Grade.grade_id == grade_id).first()

def get_student_grades(db: Session, student_id: int):
    return db.query(models.Grade).filter(models.Grade.student_id ==
student_id).all()

def create_grade(db: Session, grade: schemas.GradeCreate):
    db_grade = models.Grade(**grade.model_dump())
    db.add(db_grade)
    db.commit()
    db.refresh(db_grade)
    return db_grade

def update_grade(db: Session, grade_id: int, grade: schemas.GradeCreate):
    db_grade = get_grade(db, grade_id)
    if db_grade:
        for key, value in grade.model_dump().items():
            setattr(db_grade, key, value)
        db.commit()
        db.refresh(db_grade)
    return db_grade

def delete_grade(db: Session, grade_id: int):
    db_grade = get_grade(db, grade_id)
    if db_grade:
        db.delete(db_grade)
        db.commit()
        return True
    return False

# ========== Attendance ==========
def get_attendances(db: Session, skip: int = 0, limit: int = 100):
    return db.query(models.Attendance).offset(skip).limit(limit).all()

def get_attendance(db: Session, attendance_id: int):
    return db.query(models.Attendance).filter(models.Attendance.attendance_id ==
attendance_id).first()

def get_student_attendance(db: Session, student_id: int):
    return db.query(models.Attendance).filter(models.Attendance.student_id ==
student_id).all()

def create_attendance(db: Session, attendance: schemas.AttendanceCreate):
    db_attendance = models.Attendance(**attendance.model_dump())
    db.add(db_attendance)
    db.commit()
    db.refresh(db_attendance)
    return db_attendance

def update_attendance(db: Session, attendance_id: int, attendance:
schemas.AttendanceCreate):
    db_attendance = get_attendance(db, attendance_id)
    if db_attendance:
```

```
        for key, value in attendance.model_dump().items():
            setattr(db_attendance, key, value)
        db.commit()
        db.refresh(db_attendance)
    return db_attendance

def delete_attendance(db: Session, attendance_id: int):
    db_attendance = get_attendance(db, attendance_id)
    if db_attendance:
        db.delete(db_attendance)
        db.commit()
        return True
    return False
```

main.py

```
from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy.orm import Session
from typing import List

import models
import schemas
import crud
from database import SessionLocal, engine

models.Base.metadata.create_all(bind=engine)

app = FastAPI(title="Academic Performance API")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Students endpoints
@app.post("/students/", response_model=schemas.Student)
def create_student(student: schemas.StudentCreate, db: Session = Depends(get_db)):
    return crud.create_student(db, student)

@app.get("/students/", response_model=List[schemas.Student])
def read_students(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_students(db, skip=skip, limit=limit)

@app.get("/students/{student_id}", response_model=schemas.Student)
def read_student(student_id: int, db: Session = Depends(get_db)):
    db_student = crud.get_student(db, student_id)
    if db_student is None:
        raise HTTPException(status_code=404, detail="Student not found")
    return db_student

@app.put("/students/{student_id}", response_model=schemas.Student)
def update_student(student_id: int, student: schemas.StudentCreate, db: Session =
Depends(get_db)):
    db_student = crud.update_student(db, student_id, student)
    if db_student is None:
        raise HTTPException(status_code=404, detail="Student not found")
    return db_student

@app.delete("/students/{student_id}")
def delete_student(student_id: int, db: Session = Depends(get_db)):
    if not crud.delete_student(db, student_id):
        raise HTTPException(status_code=404, detail="Student not found")
    return {"message": "Student deleted successfully"}
```

```
# Teachers endpoints
@app.post("/teachers/", response_model=schemas.Teacher)
def create_teacher(teacher: schemas.TeacherCreate, db: Session = Depends(get_db)):
    return crud.create_teacher(db, teacher)

@app.get("/teachers/", response_model=List[schemas.Teacher])
def read_teachers(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_teachers(db, skip=skip, limit=limit)

@app.get("/teachers/{teacher_id}", response_model=schemas.Teacher)
def read_teacher(teacher_id: int, db: Session = Depends(get_db)):
    db_teacher = crud.get_teacher(db, teacher_id)
    if db_teacher is None:
        raise HTTPException(status_code=404, detail="Teacher not found")
    return db_teacher

@app.put("/teachers/{teacher_id}", response_model=schemas.Teacher)
def update_teacher(teacher_id: int, teacher: schemas.TeacherCreate, db: Session =
Depends(get_db)):
    db_teacher = crud.update_teacher(db, teacher_id, teacher)
    if db_teacher is None:
        raise HTTPException(status_code=404, detail="Teacher not found")
    return db_teacher

@app.delete("/teachers/{teacher_id}")
def delete_teacher(teacher_id: int, db: Session = Depends(get_db)):
    if not crud.delete_teacher(db, teacher_id):
        raise HTTPException(status_code=404, detail="Teacher not found")
    return {"message": "Teacher deleted successfully"}

# Subjects endpoints
@app.post("/subjects/", response_model=schemas.Subject)
def create_subject(subject: schemas.SubjectCreate, db: Session = Depends(get_db)):
    return crud.create_subject(db, subject)

@app.get("/subjects/", response_model=List[schemas.Subject])
def read_subjects(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    return crud.get_subjects(db, skip=skip, limit=limit)

@app.get("/subjects/{subject_id}", response_model=schemas.Subject)
def read_subject(subject_id: int, db: Session = Depends(get_db)):
    db_subject = crud.get_subject(db, subject_id)
    if db_subject is None:
        raise HTTPException(status_code=404, detail="Subject not found")
    return db_subject

@app.put("/subjects/{subject_id}", response_model=schemas.Subject)
def update_subject(subject_id: int, subject: schemas.SubjectCreate, db: Session =
Depends(get_db)):
    db_subject = crud.update_subject(db, subject_id, subject)
    if db_subject is None:
        raise HTTPException(status_code=404, detail="Subject not found")
    return db_subject

@app.delete("/subjects/{subject_id}")
def delete_subject(subject_id: int, db: Session = Depends(get_db)):
    if not crud.delete_subject(db, subject_id):
        raise HTTPException(status_code=404, detail="Subject not found")
    return {"message": "Subject deleted successfully"}

# Grades endpoints
# ========== Grades endpoints ==========
@app.post("/grades/", response_model=schemas.Grade,
status_code=status.HTTP_201_CREATED)
def create_grade(grade: schemas.GradeCreate, db: Session = Depends(get_db)):
```

```
    try:
        return crud.create_grade(db, grade)
    except Exception as e:
        raise HTTPException(status_code=400, detail=str(e))

@app.get("/grades/", response_model=List[schemas.Grade])
def read_grades(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
    try:
        return crud.get_grades(db, skip=skip, limit=limit)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error reading grades:
{str(e)}")

@app.get("/grades/student/{student_id}", response_model=List[schemas.Grade])
def read_student_grades(student_id: int, db: Session = Depends(get_db)):
    try:
        return crud.get_student_grades(db, student_id)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error reading student grades:
{str(e)}")

@app.get("/grades/{grade_id}", response_model=schemas.Grade)
def read_grade(grade_id: int, db: Session = Depends(get_db)):
    db_grade = crud.get_grade(db, grade_id)
    if db_grade is None:
        raise HTTPException(status_code=404, detail="Grade not found")
    return db_grade

@app.put("/grades/{grade_id}", response_model=schemas.Grade)
def update_grade(grade_id: int, grade: schemas.GradeCreate, db: Session =
Depends(get_db)):
    db_grade = crud.update_grade(db, grade_id, grade)
    if db_grade is None:
        raise HTTPException(status_code=404, detail="Grade not found")
    return db_grade

@app.delete("/grades/{grade_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_grade(grade_id: int, db: Session = Depends(get_db)):
    if not crud.delete_grade(db, grade_id):
        raise HTTPException(status_code=404, detail="Grade not found")
    return None
# Attendance endpoints
@app.post("/attendances/", response_model=schemas.Attendance)
def create_attendance(attendance: schemas.AttendanceCreate, db: Session =
Depends(get_db)):
    return crud.create_attendance(db, attendance)

@app.get("/attendances/", response_model=List[schemas.Attendance])
def read_attendances(skip: int = 0, limit: int = 100, db: Session =
Depends(get_db)):
    return crud.get_attendances(db, skip=skip, limit=limit)

@app.get("/attendances/student/{student_id}",
response_model=List[schemas.Attendance])
def read_student_attendance(student_id: int, db: Session = Depends(get_db)):
    return crud.get_student_attendance(db, student_id)

@app.get("/attendances/{attendance_id}", response_model=schemas.Attendance)
def read_attendance(attendance_id: int, db: Session = Depends(get_db)):
    db_attendance = crud.get_attendance(db, attendance_id)
    if db_attendance is None:
        raise HTTPException(status_code=404, detail="Attendance record not found")
    return db_attendance

@app.put("/attendances/{attendance_id}", response_model=schemas.Attendance)
```

```
def update_attendance(attendance_id: int, attendance: schemas.AttendanceCreate, db:
Session = Depends(get_db)):
    db_attendance = crud.update_attendance(db, attendance_id, attendance)
    if db_attendance is None:
        raise HTTPException(status_code=404, detail="Attendance record not found")
    return db_attendance

@app.delete("/attendances/{attendance_id}")
def delete_attendance(attendance_id: int, db: Session = Depends(get_db)):
    if not crud.delete_attendance(db, attendance_id):
        raise HTTPException(status_code=404, detail="Attendance record not found")
    return {"message": "Attendance record deleted successfully"}

@app.get("/")
def root():
    return {"message": "Academic Performance API is running", "docs": "/docs"}
```

run.py

```
import uvicorn

if __name__ == "__main__":
    uvicorn.run("main:app", host="127.0.0.1", port=8000, reload=True)
```

127.0.0.1:8000/docs#/

Academic Performance API 0.1.0 OAS 3.1

/openapi.json

default

POST	/students/	Create Student
GET	/students/	Read Students
GET	/students/{student_id}	Read Student
PUT	/students/{student_id}	Update Student
DELETE	/students/{student_id}	Delete Student
POST	/teachers/	Create Teacher
GET	/teachers/	Read Teachers
GET	/teachers/{teacher_id}	Read Teacher
PUT	/teachers/{teacher_id}	Update Teacher
DELETE	/teachers/{teacher_id}	Delete Teacher
POST	/subjects/	Create Subject
GET	/subjects/	Read Subjects

GET /grades/student/{student_id} Read Student Grades

GET /grades/{grade_id} Read Grade

PUT /grades/{grade_id} Update Grade

DELETE /grades/{grade_id} Delete Grade

POST /attendances/ Create Attendance

GET /attendances/ Read Attendances

GET /attendances/student/{student_id} Read Student Attendance

GET /attendances/{attendance_id} Read Attendance

PUT /attendances/{attendance_id} Update Attendance

DELETE /attendances/{attendance_id} Delete Attendance

GET / Root

Schemas

Attendance ^ Collapse all object

```
student_id*  integer
subject_id*  integer
attendance_date*  string  date
status*  string
attendance_id*  integer
```

AttendanceCreate > Expand all object

Grade > Expand all object

GradeCreate > Expand all object

HTTPValidationError > Expand all object

Student > Expand all object

StudentCreate > Expand all object

Subject > Expand all object

SubjectCreate > Expand all object

Teacher > Expand all object

TeacherCreate > Expand all object

GET /grades/ Read Grades

Parameters

Cancel

Name	Description
skip integer (query)	0
limit integer (query)	100

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/grades/?skip=0&limit=100' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/grades/?skip=0&limit=100
```

Server response

Code	Details
200	Response body


```
[
  {
    "student_id": 4,
    "subject_id": 10,
    "grade": 5,
    "grade_date": "2024-01-15",
    "semester": 2,
    "grade_id": 38
```

```
  {
    "student_id": 5,
    "subject_id": 10,
    "grade": 5,
    "grade_date": "2024-01-15",
    "semester": 2,
    "grade_id": 40
  },
  {
    "student_id": 5,
```


Download

Response headers


```
content-length: 673
content-type: application/json
date: Thu,26 Mar 2026 20:58:56 GMT
server: uvicorn
```


Responses

Code	Description	Links
200	Successful Response	*No links*

Media type

application/json

Controls Accept header.

Example Value | Schema


```
[
  {
    "student_id": 0,
    "subject_id": 0,
    "grade": 0,
    "grade_date": "2026-03-26",
    "semester": 0,
    "grade_id": 0
  }
]
```


Вывод: были отработаны навыки разработки API и баз данных.